A SIMD-Capable Execution Stage: Design Exploration and Characterisation

MEL G624 — Advanced VLSI Architectures, Semester I 2026–27 Assignment 1

Name	_____
ID No.	_____
Date of issue	24 October 2026
Date of submission	3 November 2026

1. Specification and interpretation

The task is to design the execution stage of a SIMD-capable datapath. It takes two 8-bit inputs `A` and `B`, a 2-bit function select `fn`, and a 1-bit size control, and produces two 8-bit outputs `C` and `D`.

Signal	Width	Meaning
`A`, `B`	8	operands, unsigned
`fn`	2	`00` add, `01` multiply, `10` equality, `11` reserved
`size`	1	`0` = one 8-bit operand per input, `1` = two packed 4-bit operands
`C`, `D`	8 each	results

The six specified behaviours are:

`size`	`fn`	`D`	`C`
0	add	`A + B`, carry-out discarded	0
0	mul	low 8 bits of `A × B`	high 8 bits
0	eq	`D[0] = (A == B)`, rest 0	0
1	add	`D[3:0] = A[3:0]+B[3:0]`, `D[7:4] = A[7:4]+B[7:4]`, each carry discarded	0
1	mul	`A[3:0] × B[3:0]`	`A[7:4] × B[7:4]`
1	eq	`D[0] = (A[3:0]==B[3:0])`, `D[4] = (A[7:4]==B[7:4])`, rest 0	0

Assumptions made where the specification is silent. The encoding `fn = 2'b11` is not defined; all four variants drive `C = D = 0` for it, and the testbench checks that. All arithmetic is

unsigned, as stated. Neither the 8-bit nor the 4-bit multiply can overflow its 16-bit and 8-bit result fields respectively (255×255 = 65025 and 15×15 = 225), so no saturation logic is required.

---

2. The central design question

The six behaviours above could be implemented by building six independent functional units and multiplexing their outputs. That solution is correct and it is what Section 4.1 calls Variant 1 — but it is the wrong answer, because in every cycle at least five of the six units compute a result that is discarded.

The useful observation is that **each 8-bit unit already contains the two 4-bit units it would otherwise sit beside.** The design work is in finding, for each operation, the minimum modification that lets one structure serve both modes.

2.1 Addition — breaking the carry chain

An 8-bit adder differs from two independent 4-bit adders only in the carry that crosses the nibble boundary. Gating that one wire is sufficient:

```
c_mid = c_out_of_low_nibble AND (NOT size)
```

With `size = 0` the carry propagates and the structure is an 8-bit adder; with `size = 1` the high nibble sees a zero carry-in and the two halves are independent. **The entire cost of SIMD support in the adder is one AND gate.**

In the speed-optimised variant the high nibble is evaluated speculatively for both carry-in values and selected afterwards, which moves this AND gate off the critical path — it then only drives a multiplexer select line, which settles while the two candidate sums are still being computed.

2.2 Multiplication — gating the cross products

This is where the real design content lies. Write the operands in terms of their nibbles, `A = 16·A_H + A_L` and `B = 16·B_H + B_L`. Then

```
A × B = 256·(A_H·B_H) + 16·(A_H·B_L + A_L·B_H) + (A_L·B_L)
                         \_________ cross terms _________/
```

The SIMD mode needs exactly `A_L·B_L` in `D` and `A_H·B_H` in `C`, and those are precisely the two terms that survive if the cross terms are forced to zero. What remains is

```
256·(A_H·B_H) + (A_L·B_L)
```

and because `A_L·B_L ≤ 225 < 256`, the two surviving terms occupy **disjoint bit ranges**. The 16-bit product therefore degenerates into a bit concatenation:

```
p = { A_H·B_H , A_L·B_L } = { C , D }
```

This is the key result of the assignment. A single 8×8 multiplier serves both modes with **no duplicated datapath and no output multiplexer at all** — both modes read their results off the very same product wires, `D = p[7:0]` and `C = p[15:8]`, in both cases. The only cost is that the 32 partial-product AND gates belonging to cross terms gain a third input, `~size`:

```
if (cross_term(i,j))  pp[i][j] = a[i] & b[j] & ~size;
else                  pp[i][j] = a[i] & b[j];
```

Because the gating happens at the leaves of the partial-product array, the reduction network behind it is completely unaware of the mode. Any multiplier microarchitecture — ripple array, Wallace, Dadda, Booth — can be dropped in unchanged, which is what makes Variants 3 and 4 a clean comparison.

2.3 Equality — one extra AND

Nibble-wise comparison gives `eq_lo` and `eq_hi` directly; the 8-bit result is `eq_lo AND eq_hi`. The 8-bit comparator and the SIMD comparator differ by one gate.

---

3. What was explored, and why

Four design points were built and measured. They are chosen to separate two independent questions — *does sharing help?* and *does hand-structuring help?* — rather than to pile up variations.

	Sharing	Arithmetic structure
V1 naive	none — duplicated units	inferred by the synthesiser
V2 shared	full	inferred by the synthesiser
V3 structural RCA	full	ripple-carry adder, shift-and-add array multiplier
V4 structural fast	full	carry-select adder, Dadda tree + Kogge-Stone final adder

V1→V2 isolates the benefit of sharing. V2→V3 isolates what is gained or lost by taking the arithmetic away from the synthesiser. V3→V4 isolates the benefit of logarithmic-depth structures over linear ones.

All four present an identical port list, so one testbench, one constraint file and one set of synthesis runs cover every design point. `simd_exec_top` selects between them with a `VARIANT` parameter.

---

4. Verification

A single golden model was transcribed directly from the specification text (not from any implementation) and all four variants were checked against it **exhaustively**:

```
2 size modes × 4 fn encodings × 256 A values × 256 B values = 524 288
vectors
524 288 vectors × 4 variants                                  = 2 097 152
comparisons
```

```
 vectors applied : 524288  (x4 variants = 2097152 comparisons)
 mismatches      : 0
 RESULT: *** ALL VARIANTS PASS ***
```

Exhaustive verification is affordable here because the input space is only 2^19 points including the mode bits, so there is no reason to settle for random or directed testing. The reserved `fn = 2'b11` encoding is included in the sweep.

A second, short testbench (`tb/tb_simd_wave.v`) applies 24 labelled stimuli covering every mode in 250 ns, for waveform inspection. It deliberately applies the same operand pair with `size = 0` and then `size = 1` so the carry chain and the cross products can be seen being broken:

```
add  0x3F+0x11 8b   A=3f B=11 fn=00 size=0 -> C=00 D=50
add  0x3F+0x11 4b   A=3f B=11 fn=00 size=1 -> C=00 D=40   <- carry
blocked
mul  0x33*0x22 8b   A=33 B=22 fn=01 size=0 -> C=06 D=c6
mul  0x33*0x22 4b   A=33 B=22 fn=01 size=1 -> C=06 D=06   <- cross terms
killed
```

---

5. Characterisation methodology

Synthesis is by **Yosys 0.33** with **ABC** technology mapping against the **Nangate 45 nm Open Cell Library** (typical corner). Area is reported in µm² and also as gate equivalents, where 1 GE is the area of a `NAND2_X1` (0.798 µm²). Delay is the ABC static-timing critical path through the mapped netlist, in picoseconds.

Every variant is mapped **twice**, under two different optimisation targets:

- **area-oriented** — ABC `map -a`
- **delay-oriented** — ABC `map -B 0.9` followed by buffering and gate up/down-sizing

Reporting both is what makes the comparison honest. A design that wins on area under an area-oriented tool may not be the right choice once the tool is told to chase frequency, and a single-target comparison hides that.

The flow is reproducible with `make report`.

---

6. Characterisation results

6.1 Complete execution stage

Area-oriented mapping

Variant	Cells	Area (µm²)	Gate equiv.	Delay (ps)	Fmax (MHz)	Area vs V1
V1 naive behavioural	589	583.1	731	1056	947	100%
V2 shared behavioural	490	469.8	589	1075	930	81%
V3 structural RCA + array	422	**423.7**	531	1550	645	73%
V4 structural CSel + Dadda	441	438.9	550	**880**	**1137**	75%

Delay-oriented mapping

Variant	Cells	Area (µm²)	Gate equiv.	Delay (ps)	Fmax (MHz)	Area vs V1
V1 naive behavioural	773	879.1	1102	535	1868	100%
V2 shared behavioural	639	**739.7**	927	564	1774	84%
V3 structural RCA + array	726	785.5	984	946	1057	89%
V4 structural CSel + Dadda	660	780.2	978	**494**	**2023**	89%

6.2 Area–delay product

The area–delay product (ADP) is the standard single-figure merit when neither area nor speed is individually constrained.

Variant	ADP, area-opt (µm²·ps)	ADP, delay-opt (µm²·ps)
V1 naive	615 955	470 598
V2 shared	504 992	416 923
V3 structural RCA	656 712	742 847
V4 structural fast	**386 144**	**385 643**

V4 has the best ADP under both mapping targets, by a margin of 23–35 % over the next best.

6.3 Where the area and the delay actually go

Block	Area-opt area (µm²)	Area-opt delay (ps)	Delay-opt area (µm²)	Delay-opt delay (ps)
SIMD adder, ripple carry	38.6	361	71.3	228
SIMD adder, carry select	45.5	284	88.3	175
SIMD equality comparator	17.3	102	68.9	91
SIMD multiplier, shift-add array	360.7	1460	736.8	922
SIMD multiplier, Dadda + Kogge-Stone	385.2	792	740.8	454

This table is the most useful one in the report. **The multiplier is the design.** It accounts for 85–88 % of the total cell area and well over 90 % of the critical path; the adder and the comparator are rounding errors beside it.

Two consequences follow directly:

- Replacing the shift-and-add array with a Dadda tree buys a **1.84× delay reduction for a 6.8 % area increase** (1460 → 792 ps for 360.7 → 385.2 µm²). This single change accounts for essentially all of V4's advantage over V3.
- Replacing the ripple-carry adder with a carry-select adder buys 1.27× on the adder itself, but the adder is not on the critical path of the block, so the effect at the top level is negligible. **Effort spent optimising the adder in this design is close to wasted.**

---

7. Analysis and recommendation

Sharing is free money. V1 → V2 removes 19 % of the area under area-oriented mapping and 16 % under delay-oriented mapping, for a delay cost of under 2 % and no change in behaviour. There is no scenario in which the duplicated-datapath design is preferable. The entire saving comes from the multiplier: V1 instantiates an 8×8 multiplier *and* two 4×4 multipliers where V2 instantiates one 8×8 multiplier with 32 gated AND gates.

Hand-structuring is not automatically a win. V3 is fully structural and is the smallest design under area-oriented mapping (423.7 µm²), but it is also by far the slowest (1550 ps) and it has the *worst* ADP of all four. Writing ripple-carry arithmetic by hand took work and produced a design that a synthesiser beats on every axis except raw area. The lesson is that structural RTL pays off only when the structure encodes something the tool will not infer — which is exactly what V4 does.

V4 dominates. Under both mapping targets V4 is faster than every other variant *and* smaller than V1 and V2. Its only concession is 3.6 % more area than V3, in exchange for being 1.76× faster. On a strict Pareto analysis the frontier contains just two points:

- **V3** — minimum area, if area is the only thing that matters
- **V4** — everything else

V1 and V2 are strictly dominated by V4 and should not be chosen under any objective.

Which solution is optimal for which need

If the need is…	Choose	Why
Absolute minimum area, speed irrelevant	**V3**	423.7 µm², 3.6 % smaller than V4
Maximum frequency	**V4**	494 ps / 2023 MHz, delay-oriented mapping
Best overall balance (the usual case)	**V4**	best ADP under both targets, 23–35 % ahead
Shortest design time	**V2**	behavioural, ~40 lines, still 19 % smaller than naive
FPGA target	**V2 or V4**	see caveat below

The gap between V3 and V4 on area is 15.2 µm² — about 19 gate equivalents, or roughly four NAND2s per percent of frequency given up. In almost any real design that trade is worth taking, which is why V4 is the recommended implementation.

Caveat on the implementation medium

These numbers are for a 45 nm standard-cell ASIC. On an FPGA the ranking would shift: dedicated carry chains make ripple-carry structures far cheaper relatively than they are in an ASIC, which would narrow or reverse the V3–V4 gap on the adder, and LUT-based mapping changes the

relative cost of the compressor tree. The Vivado flow in this repository (`docs/VIVADO.md`) is set up to measure exactly that; the comparison is left as a separate exercise because the two media are not meaningfully comparable on one axis.

---

8. Conclusions

1. The SIMD requirement is almost free if it is designed in rather than bolted on. One AND gate breaks the adder's carry chain; 32 three-input AND gates in place of two-input ones make a single 8×8 multiplier serve both modes.
2. The partial-product decomposition is the crux: zeroing the cross terms makes the 16-bit product collapse to `{A_H·B_H, A_L·B_L}`, which is exactly the packed SIMD result, with no output multiplexer required.
3. Resource sharing saves 19 % area at under 2 % delay cost — an unambiguous win.
4. The multiplier dominates both area (85–88 %) and delay (>90 %). Design effort belongs there and essentially nowhere else.
5. The structural carry-select + Dadda design (V4) is the recommended solution: best ADP under both mapping targets, fastest overall, and within 3.6 % of the minimum-area design.

---

9. Reproducing these results

```
make lint              # elaboration check
make sim               # short waveform testbench
make sim-exhaustive    # 524 288 vectors x 4 variants
make report            # synthesis + characterisation -> results/asic/
```

For the Vivado flow on Windows, see `docs/VIVADO.md`.

Appendix A — Verilog source listings

A.1 Top level and the four variants

`rtl/simd_exec_top.v`

```
//
------------------------------------------------------------------------
---
// simd_exec_top.v : thin selector wrapper around the four design
variants.
//
// VARIANT = 1  naive behavioural          (duplicated datapath,
baseline)
//           2  shared behavioural         (gated carry + gated cross
products)
//           3  structural ripple/array    (minimum area)
//           4  structural carry-select/Dadda (maximum speed)
//
// Only the selected variant is elaborated; the generate-if prunes the
rest.
// Keeping one common top lets a single testbench, a single XDC and a
single
// set of Vivado runs cover every design point - change the parameter
(or pass
// -generic VARIANT=n to synth_design) and nothing else moves.
```

```
//
------------------------------------------------------------------------
---
`default_nettype none

module simd_exec_top #(
    parameter integer VARIANT = 4
) (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,     // 00 add, 01 multiply, 10 equality, 11
reserved
    input  wire       size,   // 0 = one 8-bit operand, 1 = two packed
4-bit
    output wire [7:0] C,
    output wire [7:0] D
);
    generate
        if (VARIANT == 1) begin : g_v1
            simd_exec_v1_naive u_dut (
                .A(A), .B(B), .fn(fn), .size(size), .C(C), .D(D));
        end
        else if (VARIANT == 2) begin : g_v2
            simd_exec_v2_shared u_dut (
                .A(A), .B(B), .fn(fn), .size(size), .C(C), .D(D));
        end
        else if (VARIANT == 3) begin : g_v3
            simd_exec_v3_struct_rca u_dut (
                .A(A), .B(B), .fn(fn), .size(size), .C(C), .D(D));
        end
        else begin : g_v4
            simd_exec_v4_struct_fast u_dut (
                .A(A), .B(B), .fn(fn), .size(size), .C(C), .D(D));
        end
    endgenerate
endmodule

`default_nettype wire
```

rtl/simd_exec_v1_naive.v

```
//
------------------------------------------------------------------------
---
// simd_exec_v1_naive.v   --  VARIANT 1 : naive behavioural baseline
//
// MEL G624 Advanced VLSI Architectures, Assignment 1
//
// The obvious first-cut solution: build a SEPARATE functional unit for
every
// (operation, size) combination and multiplex the results.  This
instantiates
// an 8-bit adder AND two 4-bit adders AND an 8x8 multiplier AND two 4x4
// multipliers AND an 8-bit comparator AND two 4-bit comparators.
//
// It is correct, it is readable, and it is the wrong answer - it pays
for
```

```
// datapath that is idle in every cycle.  It exists here purely as the
// reference point the optimised variants are measured against.
//
-------------------------------------------------------------------------
---
`default_nettype none

module simd_exec_v1_naive (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,     // 00 add, 01 multiply, 10 equality
    input  wire       size,   // 0 = one 8-bit operand, 1 = two packed
4-bit
    output wire [7:0] C,
    output wire [7:0] D
);
    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    // ---- dedicated 8-bit units
-----------------------------------------
    wire [7:0]  add8 = A + B;            // carry-out discarded by
truncation
    wire [15:0] mul8 = A * B;
    wire        eq8  = (A == B);

    // ---- dedicated 4-bit units (duplicated hardware)
----------------------
    wire [3:0] add4_lo = A[3:0] + B[3:0];
    wire [3:0] add4_hi = A[7:4] + B[7:4];
    wire [7:0] mul4_lo = A[3:0] * B[3:0];
    wire [7:0] mul4_hi = A[7:4] * B[7:4];
    wire       eq4_lo  = (A[3:0] == B[3:0]);
    wire       eq4_hi  = (A[7:4] == B[7:4]);

    // ---- output multiplexer
-----------------------------------------------
    reg [7:0] Cr, Dr;

    always @* begin
        Cr = 8'h00;
        Dr = 8'h00;
        case ({size, fn})
            {1'b0, FN_ADD} : Dr = add8;
            {1'b0, FN_MUL} : begin Dr = mul8[7:0]; Cr = mul8[15:8]; end
            {1'b0, FN_EQ } : Dr = {7'b0, eq8};
            {1'b1, FN_ADD} : Dr = {add4_hi, add4_lo};
            {1'b1, FN_MUL} : begin Dr = mul4_lo; Cr = mul4_hi; end
            {1'b1, FN_EQ } : Dr = {3'b0, eq4_hi, 3'b0, eq4_lo};
            default        : begin Cr = 8'h00; Dr = 8'h00; end  // fn =
2'b11 reserved
        endcase
    end

    assign C = Cr;
```

```
    assign D = Dr;
endmodule

`default_nettype wire
```

rtl/simd_exec_v2_shared.v

```
//
------------------------------------------------------------------------
---
// simd_exec_v2_shared.v  --  VARIANT 2 : resource-shared behavioural
//
// Same behaviour as variant 1, but every 8-bit unit is made to CONTAIN
its
// two 4-bit counterparts instead of sitting beside them:
//
//   adder      : one 8-bit add whose nibble-crossing carry is ANDed
with ~size
//   multiplier : partial-product decomposition with the cross terms
gated off,
//                so the product degenerates to {AH*BH, AL*BL} in SIMD
mode
//   comparator : nibble compares ANDed together for the 8-bit result
//
// Still written with the +, * and == operators, so the synthesiser
remains
// free to pick its own adder and multiplier architectures.  This
variant
// isolates the benefit of SHARING from the benefit of hand-structuring.
//
------------------------------------------------------------------------
---
`default_nettype none

module simd_exec_v2_shared (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,
    input  wire       size,
    output wire [7:0] C,
    output wire [7:0] D
);
    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    wire nsize = ~size;

    // ---- shared adder: one carry chain, breakable at the nibble
boundary --
    wire [4:0] sum_lo  = {1'b0, A[3:0]} + {1'b0, B[3:0]};
    wire       c_mid   = sum_lo[4] & nsize;
    wire [3:0] sum_hi  = A[7:4] + B[7:4] + c_mid;
    wire [7:0] add_res = {sum_hi, sum_lo[3:0]};

    // ---- shared multiplier: cross partial products gated off in SIMD
mode -
```

```
    wire [7:0]  pLL  = A[3:0] * B[3:0];
    wire [7:0]  pHH  = A[7:4] * B[7:4];
    wire [7:0]  pLH  = (A[3:0] * B[7:4]) & {8{nsize}};
    wire [7:0]  pHL  = (A[7:4] * B[3:0]) & {8{nsize}};
    wire [8:0]  pmid = pLH + pHL;
    // {pHH,pLL} needs no adder: AL*BL <= 225 cannot carry into bit 8.
    wire [15:0] prod = {pHH, pLL} + {3'b0, pmid, 4'b0};

    // ---- shared comparator
-----------------------------------------------
    wire eq_lo  = (A[3:0] == B[3:0]);
    wire eq_hi  = (A[7:4] == B[7:4]);
    wire eq_all = eq_lo & eq_hi;

    wire [7:0] eq_res = size ? {3'b0, eq_hi, 3'b0, eq_lo}
                             : {7'b0,  eq_all};

    // ---- output multiplexer
-----------------------------------------------
    reg [15:0] cd;

    always @* begin
        case (fn)
            FN_ADD  : cd = {8'h00, add_res};
            FN_MUL  : cd = prod;             // serves BOTH modes
unchanged
            FN_EQ   : cd = {8'h00, eq_res};
            default : cd = 16'h0000;         // fn = 2'b11 reserved
        endcase
    end

    assign C = cd[15:8];
    assign D = cd[7:0];
endmodule

`default_nettype wire
```

rtl/simd_exec_v3_struct_rca.v

```
//
------------------------------------------------------------------------
---
// simd_exec_v3_struct_rca.v  --  VARIANT 3 : structural, area-oriented
//
// The sharing scheme of variant 2, but now every arithmetic block is
built
// explicitly out of gates and full adders rather than left to the
synthesiser:
//
//   adder      : add_simd_rca     - two 4-bit ripple chains + 1
breaking gate
//   multiplier : mult_simd_array  - gated partial products, shift-and-
add
//   comparator : eq_simd          - XNOR + AND reduction
//
// This is the minimum-area design point: ripple carry everywhere, no
// speculation, no prefix network.
```

```
//
------------------------------------------------------------------------
---
`default_nettype none

module simd_exec_v3_struct_rca (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,
    input  wire       size,
    output wire [7:0] C,
    output wire [7:0] D
);
    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    wire [7:0]  add_res;
    wire [15:0] prod;
    wire        eq_lo, eq_hi, eq_all;

    add_simd_rca u_add (
        .a(A), .b(B), .size(size), .sum(add_res)
    );

    mult_simd_array u_mul (
        .a(A), .b(B), .size(size), .p(prod)
    );

    eq_simd u_eq (
        .a(A), .b(B), .eq_lo(eq_lo), .eq_hi(eq_hi), .eq_all(eq_all)
    );

    wire [7:0] eq_res = size ? {3'b0, eq_hi, 3'b0, eq_lo}
                             : {7'b0,  eq_all};

    reg [15:0] cd;

    always @* begin
        case (fn)
            FN_ADD  : cd = {8'h00, add_res};
            FN_MUL  : cd = prod;
            FN_EQ   : cd = {8'h00, eq_res};
            default : cd = 16'h0000;
        endcase
    end

    assign C = cd[15:8];
    assign D = cd[7:0];
endmodule

`default_nettype wire

rtl/simd_exec_v4_struct_fast.v
//
------------------------------------------------------------------------
```

```
// ---------------------------------------------------------------------
// simd_exec_v4_struct_fast.v  --  VARIANT 4 : structural, speed-
oriented
//
// Identical function and identical sharing scheme to variant 3, with
every
// carry-propagate structure replaced by a logarithmic-depth one:
//
//   adder      : add_simd_csel    - carry-select high nibble, so the
gate that
//                                   breaks the carry chain sits off the
//                                   critical path (it only drives a mux
select)
//   multiplier : mult_simd_dadda  - Dadda compressor tree (35 FA + 7
HA)
//                                   followed by a 16-bit Kogge-Stone
adder
//   comparator : eq_simd          - unchanged, already a shallow AND
tree
//
// This is the maximum-speed design point.
//
// ---------------------------------------------------------------------
---
`default_nettype none

module simd_exec_v4_struct_fast (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire [1:0] fn,
    input  wire       size,
    output wire [7:0] C,
    output wire [7:0] D
);
    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    wire [7:0]  add_res;
    wire [15:0] prod;
    wire        eq_lo, eq_hi, eq_all;

    add_simd_csel u_add (
        .a(A), .b(B), .size(size), .sum(add_res)
    );

    mult_simd_dadda u_mul (
        .a(A), .b(B), .size(size), .p(prod)
    );

    eq_simd u_eq (
        .a(A), .b(B), .eq_lo(eq_lo), .eq_hi(eq_hi), .eq_all(eq_all)
    );

    wire [7:0] eq_res = size ? {3'b0, eq_hi, 3'b0, eq_lo}
                             : {7'b0,  eq_all};
```

```
    reg [15:0] cd;

    always @* begin
        case (fn)
            FN_ADD  : cd = {8'h00, add_res};
            FN_MUL  : cd = prod;
            FN_EQ   : cd = {8'h00, eq_res};
            default : cd = 16'h0000;
        endcase
    end

    assign C = cd[15:8];
    assign D = cd[7:0];
endmodule

`default_nettype wire
```

rtl/simd_exec_timed.v

```
//
------------------------------------------------------------------------
---
// simd_exec_timed.v : register-to-register wrapper used for timing
closure.
//
// The execution stage itself is pure combinational logic, so a static
timing
// tool has no clock to measure it against.  Sandwiching it between
input and
// output flops reproduces how the block actually sits in a pipeline and
lets
// Vivado (or OpenSTA) report a genuine Fmax instead of an unconstrained
path.
//
// Synthesise THIS module when you want frequency numbers; synthesise
// simd_exec_top when you want the pure combinational gate count.
//
------------------------------------------------------------------------
---
`default_nettype none

module simd_exec_timed #(
    parameter integer VARIANT = 4
) (
    input  wire       clk,
    input  wire       rst_n,
    input  wire [7:0] A_in,
    input  wire [7:0] B_in,
    input  wire [1:0] fn_in,
    input  wire       size_in,
    output reg  [7:0] C_out,
    output reg  [7:0] D_out
);
    reg  [7:0] A_q, B_q;
    reg  [1:0] fn_q;
    reg        size_q;
```

```
    wire [7:0] C_w, D_w;

    simd_exec_top #(.VARIANT(VARIANT)) u_exec (
        .A(A_q), .B(B_q), .fn(fn_q), .size(size_q), .C(C_w), .D(D_w)
    );

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            A_q    <= 8'h00;
            B_q    <= 8'h00;
            fn_q   <= 2'b00;
            size_q <= 1'b0;
            C_out  <= 8'h00;
            D_out  <= 8'h00;
        end else begin
            A_q    <= A_in;
            B_q    <= B_in;
            fn_q   <= fn_in;
            size_q <= size_in;
            C_out  <= C_w;
            D_out  <= D_w;
        end
    end
endmodule

`default_nettype wire
```

A.2 Arithmetic building blocks

rtl/lib/prim.v

```
//
------------------------------------------------------------------------
---
// prim.v : primitive 1-bit adder cells
// MEL G624 Advanced VLSI Architectures - Assignment 1
//
------------------------------------------------------------------------
---
`default_nettype none

// Full adder : 3 -> 2 compressor
module fa (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

// Half adder : 2 -> 2 compressor
module ha (
    input  wire a,
    input  wire b,
```

```
    output wire sum,
    output wire cout
);
    assign sum  = a ^ b;
    assign cout = a & b;
endmodule

`default_nettype wire
```

rtl/lib/adder_rca.v

```
//
------------------------------------------------------------------------
---
// adder_rca.v : parameterised ripple-carry adder built from full adders
//
------------------------------------------------------------------------
---
`default_nettype none

module adder_rca #(
    parameter W = 8
) (
    input  wire [W-1:0] a,
    input  wire [W-1:0] b,
    input  wire         cin,
    output wire [W-1:0] sum,
    output wire         cout
);
    wire [W:0] c;
    assign c[0] = cin;

    genvar i;
    generate
        for (i = 0; i < W; i = i + 1) begin : g_bit
            fa u_fa (
                .a    (a[i]),
                .b    (b[i]),
                .cin  (c[i]),
                .sum  (sum[i]),
                .cout (c[i+1])
            );
        end
    endgenerate

    assign cout = c[W];
endmodule

`default_nettype wire
```

rtl/lib/adder_ks.v

```
//
------------------------------------------------------------------------
---
// adder_ks.v : parameterised Kogge-Stone parallel-prefix adder
//
// Carry-in is folded into the generate term of bit 0, which is legal
```

```
because
// the prefix operator "o" is associative:
//     G[i:0] = g_i + p_i*(g_(i-1) + ... + p_1*(g_0 + p_0*cin))
// so pre-loading g_0' = g_0 + p_0*cin yields the correct carry at every
bit
// position without spending an extra prefix level on the carry-in.
//
// Logic depth = ceil(log2(W)) prefix levels + 1 XOR.
//
// The prefix network is held in flat vectors (not unpacked arrays) so
that the
// source elaborates identically in Vivado, XSim, Icarus and Yosys.
//
------------------------------------------------------------------------
---
`default_nettype none

module adder_ks #(
    parameter W = 16
) (
    input  wire [W-1:0] a,
    input  wire [W-1:0] b,
    input  wire         cin,
    output wire [W-1:0] sum,
    output wire         cout
);
    localparam integer LEVELS = (W <= 1) ? 1 : $clog2(W);

    wire [W-1:0] p0 = a ^ b;   // bit propagate (also the half sum)
    wire [W-1:0] g0 = a & b;   // bit generate

    // Level-0 prefix terms, carry-in folded into bit 0.
    wire [W-1:0] Gin;
    wire [W-1:0] Pin;
    assign Gin = g0 | ((p0 & {{(W-1){1'b0}}, cin}));
    assign Pin = p0;

    // Flat prefix network: level k occupies bits [k*W +: W].
    wire [(LEVELS+1)*W-1:0] Gn;
    wire [(LEVELS+1)*W-1:0] Pn;

    assign Gn[0 +: W] = Gin;
    assign Pn[0 +: W] = Pin;

    genvar k, i;
    generate
        for (k = 0; k < LEVELS; k = k + 1) begin : g_level
            for (i = 0; i < W; i = i + 1) begin : g_node
                if (i >= (1 << k)) begin : g_combine
                    // (G,P) o (G',P') = (G + P*G', P*P')
                    assign Gn[(k+1)*W + i] = Gn[k*W + i] |
                                             (Pn[k*W + i] & Gn[k*W + i -
(1 << k)]);
                    assign Pn[(k+1)*W + i] = Pn[k*W + i] &
                                              Pn[k*W + i - (1 << k)];
                end else begin : g_pass
```

```
                    assign Gn[(k+1)*W + i] = Gn[k*W + i];
                    assign Pn[(k+1)*W + i] = Pn[k*W + i];
                end
            end
        end
    endgenerate

    // Carry into bit i is the group generate spanning bits [i-1:0].
    wire [W-1:0] cbit;
    assign cbit[0] = cin;
    generate
        for (i = 1; i < W; i = i + 1) begin : g_carry
            assign cbit[i] = Gn[LEVELS*W + i - 1];
        end
    endgenerate

    assign sum  = p0 ^ cbit;
    assign cout = Gn[LEVELS*W + W - 1];
endmodule

`default_nettype wire
```

rtl/lib/add_simd_rca.v

```
//
------------------------------------------------------------------------
---
// add_simd_rca.v : SIMD-breakable 8-bit adder, ripple-carry flavour.
//
//   size = 0 -> one 8-bit add,  sum = A + B      (carry-out discarded)
//   size = 1 -> two 4-bit adds, sum = {A[7:4]+B[7:4], A[3:0]+B[3:0]}
//
// The ENTIRE cost of SIMD support here is the single AND gate on the
carry
// that crosses the nibble boundary.  Nothing else is duplicated.
//
------------------------------------------------------------------------
---
`default_nettype none

module add_simd_rca (
    input  wire [7:0] a,
    input  wire [7:0] b,
    input  wire       size,
    output wire [7:0] sum
);
    wire c_lo;    // carry out of the low nibble
    wire c_mid;   // carry actually injected into the high nibble

    adder_rca #(.W(4)) u_lo (
        .a(a[3:0]), .b(b[3:0]), .cin(1'b0),
        .sum(sum[3:0]), .cout(c_lo)
    );

    // the one gate that breaks the carry chain
    assign c_mid = c_lo & ~size;
```

```
    adder_rca #(.W(4)) u_hi (
        .a(a[7:4]), .b(b[7:4]), .cin(c_mid),
        .sum(sum[7:4]), .cout(/* discarded per spec */)
    );
endmodule

`default_nettype wire
```

rtl/lib/add_simd_csel.v

```
//
------------------------------------------------------------------------
---
// add_simd_csel.v : SIMD-breakable 8-bit adder, carry-select flavour.
//
// Same function as add_simd_rca but the high nibble is evaluated for
both
// possible carry-ins in parallel and selected afterwards.  This keeps
the
// nibble-breaking AND gate OFF the critical path: it only feeds the
select
// line of the output mux, which settles while the two speculative
nibble
// sums are still being computed.
//
------------------------------------------------------------------------
---
`default_nettype none

module add_simd_csel (
    input  wire [7:0] a,
    input  wire [7:0] b,
    input  wire       size,
    output wire [7:0] sum
);
    wire       c_lo;
    wire [3:0] hi_c0, hi_c1;
    wire       c_mid;

    adder_ks #(.W(4)) u_lo (
        .a(a[3:0]), .b(b[3:0]), .cin(1'b0),
        .sum(sum[3:0]), .cout(c_lo)
    );

    // speculative high-nibble sums, computed in parallel with the low
nibble
    adder_ks #(.W(4)) u_hi0 (
        .a(a[7:4]), .b(b[7:4]), .cin(1'b0), .sum(hi_c0), .cout()
    );
    adder_ks #(.W(4)) u_hi1 (
        .a(a[7:4]), .b(b[7:4]), .cin(1'b1), .sum(hi_c1), .cout()
    );

    assign c_mid    = c_lo & ~size;
    assign sum[7:4] = c_mid ? hi_c1 : hi_c0;
endmodule
```

```
`default_nettype wire
```

rtl/lib/eq_simd.v

```
//
------------------------------------------------------------------------
---
// eq_simd.v : nibble-wise equality comparator.
//
// The 8-bit compare is just the AND of the two nibble compares, so the
SIMD
// version costs one extra AND gate over a plain 8-bit comparator.
//
------------------------------------------------------------------------
---
`default_nettype none

module eq_simd (
    input  wire [7:0] a,
    input  wire [7:0] b,
    output wire       eq_lo,   // A[3:0] == B[3:0]
    output wire       eq_hi,   // A[7:4] == B[7:4]
    output wire       eq_all   // A == B
);
    wire [7:0] xnor_ab = ~(a ^ b);

    assign eq_lo  = &xnor_ab[3:0];
    assign eq_hi  = &xnor_ab[7:4];
    assign eq_all = eq_lo & eq_hi;
endmodule

`default_nettype wire
```

rtl/lib/mult_simd_array.v

```
//
------------------------------------------------------------------------
---
// mult_simd_array.v : 8x8 SIMD-splittable multiplier, shift-and-add
array.
//
// THE KEY IDEA OF THIS ASSIGNMENT
// -------------------------------
// Write A = 16*AH + AL and B = 16*BH + BL.  Then
//
//     A*B = 256*(AH*BH) + 16*(AH*BL + AL*BH) + (AL*BL)
//                          \_______ cross terms _______/
//
// If the two cross terms are forced to zero, what is left is
//
//     256*(AH*BH) + (AL*BL)
//
// and because AL*BL <= 225 < 256 the two surviving terms occupy
DISJOINT bit
// ranges.  The 16-bit product therefore degenerates to the bit
concatenation
//
//     p = { AH*BH , AL*BL }  =  { C , D }
```

```
//
// which is exactly the packed 4-bit SIMD result the specification asks
for.
//
// So a single 8x8 multiplier serves both modes with no duplicated
datapath and
// no output multiplexer: the only cost is that the 32 partial-product
AND
// gates belonging to cross terms gain a third input, ~size.  Both modes
read
// their results off the very same product wires.
//
-------------------------------------------------------------------------
---
`default_nettype none

module mult_simd_array (
    input  wire [7:0]  a,
    input  wire [7:0]  b,
    input  wire        size,    // 0 = 8x8, 1 = two independent 4x4
    output wire [15:0] p
);
    wire nsize = ~size;

    // ---- gated partial products: ppf[j*8 + i] = a[i] & b[j]
-----------------
    wire [63:0] ppf;

    genvar i, j;
    generate
        for (j = 0; j < 8; j = j + 1) begin : g_pp_row
            for (i = 0; i < 8; i = i + 1) begin : g_pp_bit
                if (((i < 4) && (j >= 4)) || ((i >= 4) && (j < 4)))
begin : g_cross
                    // cross term - killed in 4-bit SIMD mode
                    assign ppf[j*8 + i] = a[i] & b[j] & nsize;
                end else begin : g_native
                    // belongs to AL*BL or AH*BH - always live
                    assign ppf[j*8 + i] = a[i] & b[j];
                end
            end
        end
    endgenerate

    // ---- accumulation
-------------------------------------------------
    // acc_j (9 bits) = (acc_{j-1} >> 1) + pp_row_j ,  p[j] = acc_j[0]
    // One 8-bit ripple-carry adder per row; seven adders in total.
    wire [8*9-1:0] accf;

    assign accf[0 +: 9] = {1'b0, ppf[0 +: 8]};   // row 0 needs no adder

    generate
        for (j = 1; j < 8; j = j + 1) begin : g_row
            wire [7:0] rsum;
            wire       rcout;
```

```
            adder_rca #(.W(8)) u_row (
                .a    (accf[(j-1)*9 + 1 +: 8]),   // acc_{j-1} >> 1
                .b    (ppf[j*8 +: 8]),
                .cin  (1'b0),
                .sum  (rsum),
                .cout (rcout)
            );
            assign accf[j*9 +: 9] = {rcout, rsum};
        end
    endgenerate

    generate
        for (j = 0; j < 8; j = j + 1) begin : g_p_low
            assign p[j] = accf[j*9];
        end
    endgenerate

    assign p[15:8] = accf[7*9 + 1 +: 8];
endmodule

`default_nettype wire
```

A.3 Dadda-tree multiplier (generated)

rtl/lib/mult_simd_dadda.v *(excerpt — 161 lines total)*

```
//
// ------------------------------------------------------------------------
// mult_simd_dadda.v - GENERATED by scripts/gen_dadda.py - DO NOT EDIT
//
// 8x8 SIMD-splittable multiplier with a Dadda reduction tree and a
// Kogge-Stone final adder.  Partial-product gating is identical to
// mult_simd_array.v: the 32 cross-term AND gates take ~size as a third
// input, which collapses the product to {AH*BH, AL*BL} in SIMD mode.
//
// Reduction schedule : heights 8 -> 6 -> 4 -> 3 -> 2
// Compressors used   : 35 full adders, 7 half adders
//
// ------------------------------------------------------------------------
`default_nettype none

module mult_simd_dadda (
    input  wire [7:0]  a,
    input  wire [7:0]  b,
    input  wire        size,    // 0 = 8x8, 1 = two independent 4x4
    output wire [15:0] p
);
    wire nsize = ~size;

    // ---- gated partial products ---------------------------------
    wire [63:0] ppf;
    assign ppf[ 0] = a[0] & b[0];
    assign ppf[ 1] = a[1] & b[0];
    assign ppf[ 2] = a[2] & b[0];
    assign ppf[ 3] = a[3] & b[0];
```

```
    assign ppf[ 4] = a[4] & b[0] & nsize;
    assign ppf[ 5] = a[5] & b[0] & nsize;
    assign ppf[ 6] = a[6] & b[0] & nsize;
    assign ppf[ 7] = a[7] & b[0] & nsize;
    assign ppf[ 8] = a[0] & b[1];
    assign ppf[ 9] = a[1] & b[1];
    assign ppf[10] = a[2] & b[1];
    assign ppf[11] = a[3] & b[1];
    assign ppf[12] = a[4] & b[1] & nsize;
    assign ppf[13] = a[5] & b[1] & nsize;
    assign ppf[14] = a[6] & b[1] & nsize;
    assign ppf[15] = a[7] & b[1] & nsize;
    assign ppf[16] = a[0] & b[2];
    assign ppf[17] = a[1] & b[2];
    assign ppf[18] = a[2] & b[2];
    assign ppf[19] = a[3] & b[2];
    assign ppf[20] = a[4] & b[2] & nsize;
    assign ppf[21] = a[5] & b[2] & nsize;
    assign ppf[22] = a[6] & b[2] & nsize;
    assign ppf[23] = a[7] & b[2] & nsize;
    assign ppf[24] = a[0] & b[3];
    assign ppf[25] = a[1] & b[3];
    assign ppf[26] = a[2] & b[3];
    assign ppf[27] = a[3] & b[3];
    assign ppf[28] = a[4] & b[3] & nsize;
    assign ppf[29] = a[5] & b[3] & nsize;
    assign ppf[30] = a[6] & b[3] & nsize;
    assign ppf[31] = a[7] & b[3] & nsize;
    assign ppf[32] = a[0] & b[4] & nsize;
    assign ppf[33] = a[1] & b[4] & nsize;
    assign ppf[34] = a[2] & b[4] & nsize;
    assign ppf[35] = a[3] & b[4] & nsize;
    assign ppf[36] = a[4] & b[4];

    // ... 101 further generated lines omitted ...
    // (the complete file is in the repository; it is
    //  produced by scripts/gen_dadda.py, listed below)
```

scripts/gen_dadda.py

```
#!/usr/bin/env python3
"""
gen_dadda.py - emit rtl/lib/mult_simd_dadda.v

Generates an 8x8 SIMD-splittable multiplier that uses a Dadda reduction
tree
instead of a ripple array.  The partial-product gating scheme is
identical to
mult_simd_array.v (see the long comment in that file); only the
summation
network differs, so the two modules are drop-in interchangeable and give
a
clean "same function, different microarchitecture" comparison point.

Run:  python3 scripts/gen_dadda.py
"""
```

```
N  = 8          # operand width
NP = 2 * N      # product width (16)

fa_count = 0
ha_count = 0
wires    = []
body     = []
_uid     = 0


def new_wire(prefix):
    global _uid
    _uid += 1
    name = "%s%d" % (prefix, _uid)
    wires.append(name)
    return name


def is_cross(i, j):
    """True for partial products belonging to AH*BL or AL*BH."""
    return ((i < N // 2) and (j >= N // 2)) or ((i >= N // 2) and (j < N
// 2))


# ------------------------------------------------------------------
columns ---
# cols[w] = list of signal names with binary weight 2**w
cols = [[] for _ in range(NP + 2)]
for j in range(N):
    for i in range(N):
        cols[i + j].append("ppf[%d]" % (j * N + i))

max_h = max(len(c) for c in cols)

# Dadda target-height sequence 2, 3, 4, 6, 9, ...
dseq = [2]
while dseq[-1] < max_h:
    dseq.append(dseq[-1] * 3 // 2)
stages = list(reversed(dseq[:-1]))          # e.g. max_h=8 -> [6, 4, 3,
2]

# ------------------------------------------------------------------
reduction ---
for sidx, target in enumerate(stages):
    body.append("")
    body.append("    // ---- Dadda stage %d : reduce every column to
height %d"
                % (sidx + 1, target))
    for w in range(NP + 1):
        while len(cols[w]) > target:
            excess = len(cols[w]) - target
            if excess >= 2 and len(cols[w]) >= 3:
                x, y, z = cols[w].pop(0), cols[w].pop(0), cols[w].pop(0)
                s, c = new_wire("fs"), new_wire("fc")
                body.append(
                    "    fa u_fa_%-3d
```

```
(.a(%s), .b(%s), .cin(%s), .sum(%s), .cout(%s));"
                    % (fa_count, x, y, z, s, c))
                cols[w].append(s)
                cols[w + 1].append(c)
                fa_count += 1
            else:
                x, y = cols[w].pop(0), cols[w].pop(0)
                s, c = new_wire("hs"), new_wire("hc")
                body.append(
                    "    ha u_ha_%-3d
(.a(%s), .b(%s), .sum(%s), .cout(%s));"
                    % (ha_count, x, y, s, c))
                cols[w].append(s)
                cols[w + 1].append(c)
                ha_count += 1

# ------------------------------------------------------------ final
rows ---
for w in range(NP + 2):
    if w >= NP:
        assert not cols[w], "overflow past the 16-bit product at weight
%d" % w
    else:
        assert len(cols[w]) <= 2, "column %d not reduced (height %d)" %
(w, len(cols[w]))

row0 = [cols[w][0] if len(cols[w]) > 0 else "1'b0" for w in range(NP)]
row1 = [cols[w][1] if len(cols[w]) > 1 else "1'b0" for w in range(NP)]

# ------------------------------------------------------------------ emit
----
out = []
out.append("// " + "-" * 73)
out.append("// mult_simd_dadda.v - GENERATED by scripts/gen_dadda.py -
DO NOT EDIT")
out.append("//")
out.append("// %dx%d SIMD-splittable multiplier with a Dadda reduction
tree and a" % (N, N))
out.append("// Kogge-Stone final adder.  Partial-product gating is
identical to")
out.append("// mult_simd_array.v: the %d cross-term AND gates take ~size
as a third" % (N * N // 2))
out.append("// input, which collapses the product to {AH*BH, AL*BL} in
SIMD mode.")
out.append("//")
out.append("// Reduction schedule : heights %s" % (" -> ".join(str(x)
for x in [max_h] + stages)))
out.append("// Compressors used   : %d full adders, %d half adders" %
(fa_count, ha_count))
out.append("// " + "-" * 73)
out.append("`default_nettype none")
out.append("")
out.append("module mult_simd_dadda (")
out.append("    input  wire [%d:0]  a," % (N - 1))
out.append("    input  wire [%d:0]  b," % (N - 1))
out.append("    input  wire        size,    // 0 = %dx%d, 1 = two
```

```
independent %dx%d"
           % (N, N, N // 2, N // 2))
out.append("    output wire [%d:0] p" % (NP - 1))
out.append(");")
out.append("    wire nsize = ~size;")
out.append("")
out.append("    // ---- gated partial products
-------------------------------------")
out.append("    wire [%d:0] ppf;" % (N * N - 1))
for j in range(N):
    for i in range(N):
        idx = j * N + i
        if is_cross(i, j):
            out.append("    assign ppf[%2d] = a[%d] & b[%d] & nsize;" %
(idx, i, j))
        else:
            out.append("    assign ppf[%2d] = a[%d] & b[%d];" % (idx, i,
j))
out.append("")
out.append("    // ---- Dadda compressor tree
-------------------------------------")

# wire declarations, wrapped
DECL_PER_LINE = 8
for k in range(0, len(wires), DECL_PER_LINE):
    out.append("    wire " + ", ".join(wires[k:k + DECL_PER_LINE]) +
";")

out.extend(body)
out.append("")
out.append("    // ---- final carry-propagate addition of the two
surviving rows ---")
out.append("    wire [%d:0] row0 = {%s};" % (NP - 1, ",
".join(reversed(row0))))
out.append("    wire [%d:0] row1 = {%s};" % (NP - 1, ",
".join(reversed(row1))))
out.append("")
out.append("    adder_ks #(.W(%d)) u_final (" % NP)
out.append("        .a(row0), .b(row1), .cin(1'b0), .sum(p), .cout()")
out.append("    );")
out.append("endmodule")
out.append("")
out.append("`default_nettype wire")

with open("rtl/lib/mult_simd_dadda.v", "w") as f:
    f.write("\n".join(out) + "\n")

print("mult_simd_dadda.v: heights %s, %d FA, %d HA, %d compressor wires"
      % (" -> ".join(str(x) for x in [max_h] + stages), fa_count,
ha_count, len(wires)))
```

A.4 Verification

tb/tb_simd_exhaustive.v

```
//
------------------------------------------------------------------------
```

```
// ---------------------------------------------------------------------
// tb_simd_exhaustive.v : exhaustive self-checking testbench.
//
// Drives all 2^16 operand pairs through all 3 operations in both size
modes
// (393 216 vectors) into all four variants simultaneously and compares
each
// against a golden model written directly from the specification text.
//
// The reserved encoding fn = 2'b11 is also swept; the agreed behaviour
is
// C = D = 0 (documented in docs/REPORT.md, since the spec leaves it
open).
//
// Icarus :  iverilog -g2012 -o sim rtl/lib/*.v rtl/*.v
tb/tb_simd_exhaustive.v
//           vvp sim
// Vivado :  set this module as the simulation top of sim_exhaustive
//
// ---------------------------------------------------------------------
---
`timescale 1ns / 1ps
`default_nettype none

module tb_simd_exhaustive;

    reg  [7:0] A, B;
    reg  [1:0] fn;
    reg        size;

    wire [7:0] C1, D1, C2, D2, C3, D3, C4, D4;

    simd_exec_top #(.VARIANT(1)) u1 (.A(A), .B(B), .fn(fn), .size(size),
.C(C1), .D(D1));
    simd_exec_top #(.VARIANT(2)) u2 (.A(A), .B(B), .fn(fn), .size(size),
.C(C2), .D(D2));
    simd_exec_top #(.VARIANT(3)) u3 (.A(A), .B(B), .fn(fn), .size(size),
.C(C3), .D(D3));
    simd_exec_top #(.VARIANT(4)) u4 (.A(A), .B(B), .fn(fn), .size(size),
.C(C4), .D(D4));

    integer errors;
    integer vectors;
    integer ia, ib, ifn, isz;

    reg [7:0] gC, gD;

    //
-----------------------------------------------------------------------
    // Golden model - transcribed from the assignment specification, not
from
    // any of the implementations.
    //
-----------------------------------------------------------------------
    task golden;
        input  [7:0] a;
```

```
        input  [7:0] b;
        input  [1:0] f;
        input        s;
        output [7:0] oC;
        output [7:0] oD;
        reg    [15:0] full;
        begin
            oC   = 8'h00;
            oD   = 8'h00;
            full = a * b;
            case ({s, f})
                // ---- 8-bit mode
----------------------------------------
                // "D contains the sum bits after discarding the carry-
out,
                //  and all bits of C are zeros"
                3'b0_00 : oD = a + b;
                // "D contains the lower significant bits of the result
and C
                //  contains the higher significant bits"
                3'b0_01 : begin oD = full[7:0]; oC = full[15:8]; end
                // "lsb of D reflects the result of comparison while
other
                //  bits of D and all the bits of C are zeros"
                3'b0_10 : oD = {7'b0, (a == b)};

                // ---- 4-bit packed mode
-----------------------------------
                // "D[3:0] contain the sum bits of A[3:0]+B[3:0] and
D[7:4]
                //  contain the sum bits of A[7:4]+B[7:4]"
                3'b1_00 : oD = {(a[7:4] + b[7:4]), (a[3:0] + b[3:0])};
                // "D contains the multiplication of lower nibbles and C
                //  contains the multiplication of upper nibbles"
                3'b1_01 : begin oD = a[3:0] * b[3:0]; oC = a[7:4] *
b[7:4]; end
                // "bit 0 and bit 4 of D reflect the equality tests of
lower
                //  and upper nibbles, remaining bits zero"
                3'b1_10 : oD = {3'b0, (a[7:4] == b[7:4]),
                                3'b0, (a[3:0] == b[3:0])};

                default : begin oC = 8'h00; oD = 8'h00; end   // fn =
2'b11
            endcase
        end
    endtask

    task check;
        input [8*8-1:0] name;
        input [7:0]     gotC;
        input [7:0]     gotD;
        begin
            if (gotC !== gC || gotD !== gD) begin
                if (errors < 20)
                    $display("FAIL %0s  A=%02h B=%02h fn=%b size=%b :
```

```
got C=%02h D=%02h  exp C=%02h D=%02h",
                                name, A, B, fn, size, gotC, gotD, gC, gD);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        errors  = 0;
        vectors = 0;


$display("============================================================");
        $display(" Exhaustive verification : 4 variants x 2^16 pairs x 8
modes");

$display("============================================================");

        for (isz = 0; isz < 2; isz = isz + 1) begin
            for (ifn = 0; ifn < 4; ifn = ifn + 1) begin
                for (ia = 0; ia < 256; ia = ia + 1) begin
                    for (ib = 0; ib < 256; ib = ib + 1) begin
                        A    = ia;      // truncates to 8 bits
                        B    = ib;
                        fn   = ifn;
                        size = isz;
                        #1;

                        golden(A, B, fn, size, gC, gD);

                        check("V1-naive ", C1, D1);
                        check("V2-shared", C2, D2);
                        check("V3-struct", C3, D3);
                        check("V4-fast  ", C4, D4);

                        vectors = vectors + 1;
                    end
                end
            end
            $display("  ... size=%0d sweep complete (%0d vectors so far,
%0d errors)",
                     isz, vectors, errors);
        end


$display("------------------------------------------------------------");
        $display(" vectors applied : %0d  (x4 variants = %0d
comparisons)",
                 vectors, vectors * 4);
        $display(" mismatches      : %0d", errors);
        if (errors == 0)
            $display(" RESULT: *** ALL VARIANTS PASS ***");
        else
            $display(" RESULT: *** %0d FAILURES ***", errors);

$display("============================================================");
```

```
        if (errors != 0) $fatal(1, "verification failed");
        $finish;
    end

endmodule

`default_nettype wire
```

tb/tb_simd_wave.v

```
//
------------------------------------------------------------------------
---
// tb_simd_wave.v : short, waveform-oriented testbench.
//
// Purpose is to produce a SHORT, READABLE waveform in the Vivado
simulator
// (or GTKWave) that demonstrates every operating mode, rather than to
prove
// correctness - tb_simd_exhaustive.v does that.
//
// Each stimulus is held for 10 ns and labelled in the transcript, and
the
// directed sequence deliberately applies the SAME operand pair with
size=0
// and then size=1 so the SIMD split is visible side by side in the
waveform.
//
// Default simulation top for the Vivado project.
//
------------------------------------------------------------------------
---
`timescale 1ns / 1ps
`default_nettype none

module tb_simd_wave;

    localparam integer STEP = 10;

    localparam [1:0] FN_ADD = 2'b00,
                     FN_MUL = 2'b01,
                     FN_EQ  = 2'b10;

    reg  [7:0] A, B;
    reg  [1:0] fn;
    reg        size;

    // all four variants run side by side so the waveform shows them
agreeing
    wire [7:0] C1, D1, C2, D2, C3, D3, C4, D4;

    simd_exec_top #(.VARIANT(1)) u1 (.A(A), .B(B), .fn(fn), .size(size),
.C(C1), .D(D1));
    simd_exec_top #(.VARIANT(2)) u2 (.A(A), .B(B), .fn(fn), .size(size),
.C(C2), .D(D2));
    simd_exec_top #(.VARIANT(3)) u3 (.A(A), .B(B), .fn(fn), .size(size),
```

```
.C(C3), .D(D3));
    simd_exec_top #(.VARIANT(4)) u4 (.A(A), .B(B), .fn(fn), .size(size),
.C(C4), .D(D4));

    integer errors;

    // ------------------------------------------------------------------
    task apply;
        input [8*24-1:0] label;
        input [7:0]      a_in;
        input [7:0]      b_in;
        input [1:0]      f_in;
        input            s_in;
        begin
            A = a_in; B = b_in; fn = f_in; size = s_in;
            #STEP;
            $display("%-24s A=%02h B=%02h fn=%b size=%b -> C=%02h D=
%02h",
                     label, A, B, fn, size, C4, D4);
            if ({C1,D1} !== {C4,D4} || {C2,D2} !== {C4,D4} || {C3,D3} !
== {C4,D4}) begin
                $display("   *** VARIANT DISAGREEMENT: v1=%02h_%02h v2=
%02h_%02h v3=%02h_%02h v4=%02h_%02h",
                         C1,D1, C2,D2, C3,D3, C4,D4);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
`ifdef DUMP_VCD
        $dumpfile("tb_simd_wave.vcd");
        $dumpvars(0, tb_simd_wave);
`endif
        errors = 0;
        A = 0; B = 0; fn = 0; size = 0;
        #STEP;

        $display("");
        $display("=== 8-BIT MODE (size=0)
============================");
        apply("add  8b  no carry",   8'h12, 8'h34, FN_ADD, 1'b0);  //
D=46
        apply("add  8b  carry out",  8'hF0, 8'h20, FN_ADD, 1'b0);  //
D=10, carry lost
        apply("add  8b  max",        8'hFF, 8'hFF, FN_ADD, 1'b0);  //
D=FE
        apply("mul  8b  small",      8'h03, 8'h05, FN_MUL, 1'b0);  //
C=00 D=0F
        apply("mul  8b  max",        8'hFF, 8'hFF, FN_MUL, 1'b0);  //
C=FE D=01
        apply("mul  8b  0x80*0x02",  8'h80, 8'h02, FN_MUL, 1'b0);  //
C=01 D=00
        apply("eq   8b  equal",      8'h5A, 8'h5A, FN_EQ,  1'b0);  //
D=01
        apply("eq   8b  differ",     8'h5A, 8'h5B, FN_EQ,  1'b0);  //
```

```
D=00

        $display("");
        $display("=== 4-BIT PACKED MODE (size=1)
======================");
        apply("add  4b  no carry",   8'h12, 8'h34, FN_ADD, 1'b1);  //
D=46
        apply("add  4b  lo carry",   8'h1F, 8'h11, FN_ADD, 1'b1);  //
D=20 (no bleed)
        apply("add  4b  both carry", 8'hFF, 8'hFF, FN_ADD, 1'b1);  //
D=EE
        apply("mul  4b  3x5 / 2x4",  8'h23, 8'h45, FN_MUL, 1'b1);  //
C=08 D=0F
        apply("mul  4b  max",        8'hFF, 8'hFF, FN_MUL, 1'b1);  //
C=E1 D=E1
        apply("eq   4b  both equal", 8'h77, 8'h77, FN_EQ,  1'b1);  //
D=11
        apply("eq   4b  lo only",    8'h37, 8'h57, FN_EQ,  1'b1);  //
D=01
        apply("eq   4b  hi only",    8'h73, 8'h75, FN_EQ,  1'b1);  //
D=10

        $display("");
        $display("=== SAME OPERANDS, SIZE TOGGLED
======================");
        $display("    (shows the carry chain and the cross products");
        $display("     being broken by the size input)");
        apply("add  0x3F+0x11 8b",   8'h3F, 8'h11, FN_ADD, 1'b0);  //
D=50
        apply("add  0x3F+0x11 4b",   8'h3F, 8'h11, FN_ADD, 1'b1);  //
D=40
        apply("mul  0x33*0x22 8b",   8'h33, 8'h22, FN_MUL, 1'b0);  //
C=06 D=C6
        apply("mul  0x33*0x22 4b",   8'h33, 8'h22, FN_MUL, 1'b1);  //
C=06 D=06

        $display("");
        $display("=== RESERVED ENCODING fn=2'b11
======================");
        apply("reserved 8b",         8'hAA, 8'h55, 2'b11,  1'b0);  //
C=00 D=00
        apply("reserved 4b",         8'hAA, 8'h55, 2'b11,  1'b1);  //
C=00 D=00

        $display("");
        if (errors == 0)
            $display("tb_simd_wave: all four variants agree on every
stimulus.");
        else
            $display("tb_simd_wave: *** %0d DISAGREEMENTS ***", errors);
        $display("");

        #(STEP*2);
        $finish;
    end
```

```
endmodule

`default_nettype wire
```